INCIDENT REPORT

A Laptop Rented to Strangers, and the Hunt That Found Two More

Proxyware on company endpoints: proving the exit-node behavior with volume data, correcting my own first report, and turning one case into a repeatable hunt

Item	Detail
Case	Proxyware (bandwidth-sharing PUA) on company laptops, three cases (fictionalized portfolio edition)
Case 1	`RestDock.exe` on NB-OPS-118 (user `anton.p`), plus a second laptop NB-HR-042 (user `maya.l`) found during scoping
Hunt 2	`relaynode_agent.exe` on NB-SLS-305, installed silently by a free codec pack
Hunt 3	`ClipCast.exe`, a screen recorder from the official Windows app store, on NB-CC-077
Detection	Cortex XDR behavioral alerts (Command and Control category), found through manual hunting in the SIEM
Report date	Day 30
Prepared by	Tri Wibowo Cahyo, Security Analyst
Verdict	True Positive, PUA / proxyware. Medium (Case 1, Hunt 3) and High (Hunt 2). Removed from all affected endpoints
Classification	Portfolio edition. Identifiers, product names, ports, counts and dates are fictionalized

Table of Contents

1. Executive Summary 3
2. Scope and Evidence 4
3. Timeline 4
4. Case 1: The Alert 5
5. Case 1: Testing My Own First Report 5
6. Case 1: Proving the Process Owns the Traffic 6
7. Case 1: Volume, the Exit-Node Proof 7
8. Case 1: The Second Laptop 8
9. Hunt 2: Bundled in a Free Installer 8
10. Hunt 3: Straight from the App Store 9
11. The Pattern Across Three Cases 10
12. Verdict, Containment and Recommendations 10
13. Indicators and ATT&CK Mapping 11
14. Limitations and Lessons Learned 12

> **About this edition.** This report is based on real investigations I led as a SOC analyst. For portfolio use, the organization, hostnames, usernames, IP addresses, product and vendor names, ports, data volumes, counts and dates are fictionalized, and volumes are rounded. The investigation method, the tool behavior and the reasoning are kept as they happened. Dates are counted from the first alert (Day 0).

1. Executive Summary

An EDR alert labeled "Command and Control" pointed at a program called `RestDock.exe` on an operations laptop. It was not an attacker's implant. It was **proxyware**: software that rents out the laptop's internet connection to a commercial proxy network, so strangers can browse the web through it and appear to come from the company's public IP address.

My first report reached the right verdict for partly wrong reasons. When I pulled 30 days of EDR data to check it, several of its claims did not survive. The main traffic was not TCP port 80 to dozens of countries; it was **UDP to a tight cluster of hosting servers in one region**. The traffic spread across many countries was mostly failed connections. And it was not one laptop: **a second laptop had been doing the same thing for about two weeks without triggering a single alert**.

The proof that settled it was volume. Over 30 days the two laptops sent about **25 GB and received about 2.5 GB**, a ratio of roughly 10 to 1. A work laptop downloads far more than it uploads. A laptop that forwards other people's web pages does the opposite.

I then turned the case into a hunt. Two more proxyware programs followed in the next four weeks: one hidden inside a free codec pack installer, and one inside a working screen recorder from the official app store. All three are removed.

Key findings

Aspect	Finding
What	Three proxyware programs on four company laptops over about a month, each turning the laptop into an exit node for a third-party proxy network
Proof of function	Outbound volume about 10 times inbound on Case 1, and mobile-phone browser traffic leaving a Windows laptop in Hunt 2
Silent spread	The second Case 1 laptop sent about 4.5 GB and never raised a detection
Privileges	Case 1 ran and uninstalled at medium integrity: no administrator rights were needed
Detection gap	EDR alerts for these programs did not become cases in the case management platform. All three were found by hunting, not by the queue
Telemetry limit	The EDR in this environment does not keep successful TCP connect events in its raw dataset, and samples network events. Attribution needed a different data view

Top recommendations

1. Make sure EDR alerts in the Command and Control category always create a case, so the next silent laptop is found by the queue and not by luck.
2. Block the three file hashes and add a hunt for "non-browser process with outbound volume far above inbound" to the team's routine.

3. Restrict software installation from the app store and from downloaded installers to approved packages.
4. Treat proxyware as a policy and exposure issue: the company IP was used by unknown third parties, which matters for reputation and abuse complaints.

2. Scope and Evidence

Case 1 is investigated in depth. Hunts 2 and 3 used the method built in Case 1 and are reported more briefly. Every case drew on the same three sources, and none of them could answer the question alone.

Source	What it gave	Limit
Cortex XDR alerts	Process name, path, signer and hash of the program that triggered the behavioral rule	One alert shows one connection. It says nothing about scope or volume
Cortex XDR raw events (XQL)	Process launches, integrity level, registry changes, DNS queries, per-connection bytes through the network story view	Successful TCP connects are not kept in the raw dataset in this environment, and network events are sampled
SIEM (QRadar)	Where the hunt started, firewall view of the traffic, 30-day history	Network events carry IP, port and time only. They cannot name the process, and cannot be filtered by file hash
File reputation	Vendor labels for each hash	Labels describe the category, not what the program did on this network

3. Timeline

Day	Event
Day -28	Earliest proxyware network activity still in retention, on NB-HR-042 (second laptop)
Day -27	Earliest proxyware network activity still in retention, on NB-OPS-118
Day -15	`RestDock` is uninstalled on NB-HR-042 from the user's desktop session. About 13 days of activity, no alert
Day 0	Cortex XDR raises a Command and Control alert for `RestDock.exe` on NB-OPS-118. I find it while hunting in the SIEM
Day 0	First report: proxyware, one laptop, rotating destinations on TCP port 80
Day 2 to 3	30-day EDR harvest. First report corrected. Second laptop found. Detection gap identified
Day 4	`RestDock` removed from NB-OPS-118. Hash blocked
Day 15	Hunt 2: `relaynode_agent.exe` on NB-SLS-305, found from a low-volume alert category in the SIEM
Day 27	Hunt 3: `ClipCast.exe` on NB-CC-077, found from an alert on a process with an ordinary name

Day	Event
Day 30	All three programs removed from every affected endpoint. This report

4. Case 1: The Alert

The alert fired because the laptop made an HTTP connection where both the client's User-Agent and the server's response header were rare across the whole tenant. The EDR files that under Command and Control by default. Its details were enough to change the question from "is this C2?" to "what is this program?".

Field	Value
Process	`C:\Users\anton.p\AppData\Local\Programs\RestDock\RestDock.exe`
Signer	Signed by a small software vendor, valid signature
Parent	`explorer.exe`, meaning it was started from the desktop, not by a script
Reputation	Flagged by a minority of vendors, consistently labeled PUA / proxy server / not-a-virus
Install path	Per-user `AppData` folder, a common choice for software that does not ask for admin rights

A signed binary from a real company, labeled "not-a-virus" by major vendors, running from the user's own folder. That profile fits grayware much better than an intrusion. The open questions were how far it reached, how long it had run, and whether it really forwarded third-party traffic or only advertised that it could.

5. Case 1: Testing My Own First Report

I wrote a first report on Day 0. Before sharing it, a review question stopped me: **was the outbound traffic in the SIEM really from `RestDock.exe`, or just from the laptop?** The SIEM sees IP addresses and ports. It cannot see processes. I had joined the two by assumption, which is exactly how I once blamed the wrong browser in an earlier case.

So I pulled 30 days of EDR data and checked every claim against it. The verdict held. Much of the reasoning behind it did not.

First report said	30-day evidence showed	Effect
One laptop affected	Two laptops, same hash, same version	Scope doubled. Second laptop had no alert
Main traffic: TCP port 80 to dozens of countries	Main traffic: UDP to about 80 servers clustered in a few hosting blocks in one region	Exit-node argument had to be rebuilt
Spread across countries proves proxying	The spread was mostly **failed** connections. Successful ones were concentrated	Argument replaced by volume (Section 7)

First report said	30-day evidence showed	Effect
The SIEM list was filtered by file hash	SIEM network events cannot be filtered by hash	Wording fixed. Attribution redone in the EDR
First seen = install date	That date was the oldest launch still in retention. Network activity started two days earlier	Written as a lower bound, not an install date
No lateral movement or credential theft	True, but the first report never showed how it checked	Method added (process tree, logons, child processes)

> **Lesson:** the right verdict is not the same as a sound report. Each claim needs the query that supports it, and a claim that cannot be traced to data comes out, even if it sounds right.

6. Case 1: Proving the Process Owns the Traffic

My first attribution query on the raw EDR dataset returned almost nothing useful: UDP statistics and failed TCP attempts, but **no successful TCP connections at all**. In this environment the raw dataset does not keep successful TCP connect events. An empty result there means "not recorded", not "did not happen".

The EDR's **network story** view stitches connection records together with the process that owned them, including bytes sent and received. That gave a per-connection answer to the review question: every connection in the set below belongs to `RestDock.exe` by its hash, not to the laptop in general.

```
preset = network_story
| filter agent_hostname in ("<host-a>", "<host-b>")
| filter actor_process_image_sha256 = "<sha256>"
| comp count() as conns, sum(action_upload) as bytes_out,
       sum(action_download) as bytes_in
    by agent_hostname, action_network_protocol, action_remote_port
| sort desc bytes_out
```

The picture it gave was different from the first report:

Traffic group	Share	What it looked like
UDP to the relay cluster	Almost all bytes	About 80 servers in a handful of hosting blocks, all in one region. The same blocks were contacted thousands of times
Failed TCP connections	A few dozen	Spread across many countries on ordinary web ports. Never completed
Connectivity checks	Thousands, tiny	Short UDP sessions to a large cloud provider. Small volume, high frequency

The clustering matters. Home computers are scattered across consumer ISPs. These destinations sat in a few hosting blocks, which is what the **relay layer of a commercial proxy network** looks like: the place where customer requests are handed to the laptop and where the answers are collected.

The failed connections now read differently too. They are third-party requests the laptop was told to make and could not complete, refused by the destination or stopped at the perimeter. They show the laptop was taking instructions to contact arbitrary addresses on someone else's behalf.

The connectivity checks I left out of the indicator list. Blocking a major cloud provider to stop a health check would break legitimate services.

7. Case 1: Volume, the Exit-Node Proof

This is the strongest evidence in the report, and it replaced the "many countries" argument from the first version. The numbers come from the same network story aggregation, grouped by laptop and by whether the laptop was on the office network.

Group	Sent	Received	Ratio
NB-OPS-118 (`anton.p`)	~21 GB	~2 GB	~10 : 1
NB-HR-042 (`maya.l`)	~4.5 GB	~0.6 GB	~7 : 1
On the office network	~21 GB	~2.1 GB	~10 : 1
Off the office network	~4 GB	~0.5 GB	~8 : 1
Total, 30 days	**~25 GB**	**~2.5 GB**	**~10 : 1**

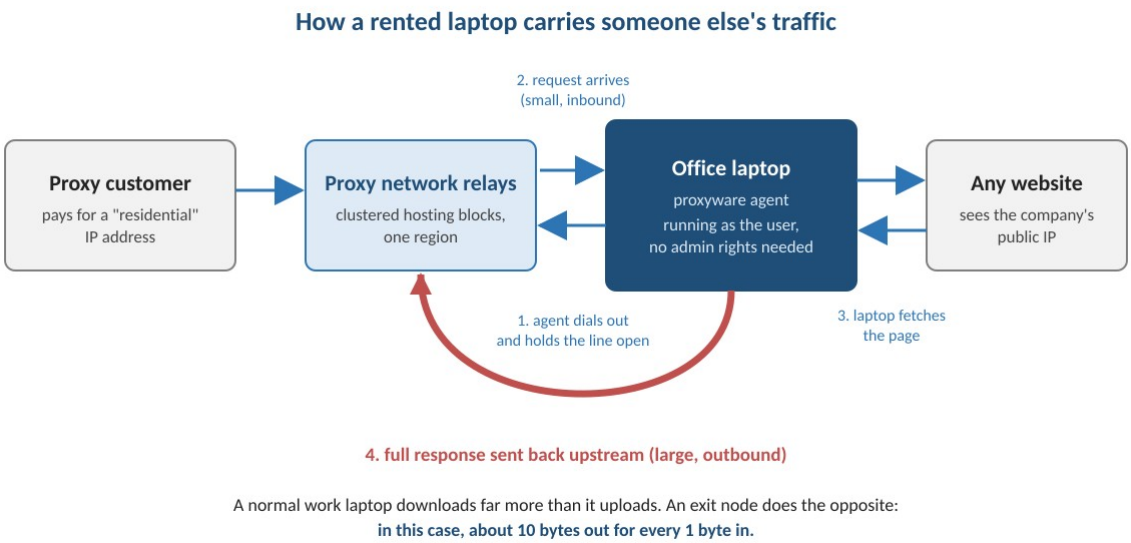


Figure 7.1 Why outbound volume is the proof: the exit-node flow

A laptop used for work receives much more than it sends. Opening pages, downloading attachments and streaming a meeting are all inbound. Here it is the reverse, by a factor of about ten. That only makes sense if the laptop is **forwarding other people's traffic**: small requests come in, full responses go back out to the proxy customer. This is a measurement, not an interpretation.

The second row group matters for risk. About **21 of the 25 GB** left while the laptops sat on the office network, so most of the third-party traffic went out under the company's own public IP address. Anything those customers did, from scraping to account abuse, would point back at the company first.

8. Case 1: The Second Laptop

The hash search across the fleet found one more laptop, NB-HR-042, belonging to a different user in a different department. Same hash, same version, nothing else anywhere.

```
dataset = xdr_data
| filter event_type = ENUM.PROCESS
| filter action_process_image_sha256 = "<sha256>"
| comp count() as launches, min(_time) as first_seen, max(_time) as last_seen
    by agent_hostname, action_process_username
| sort asc first_seen
```

NB-HR-042 had run the proxyware for about 13 days, sent about 4.5 GB, and **never produced a single detection**. Its activity stopped on Day -15, when the program was uninstalled from the user's desktop session. I found the uninstall only because the uninstaller removed an autostart value from the registry on its way out, which also confirmed how the program persisted.

8.1 Did it need admin rights?

The environment runs least privilege, and installs normally go through IT. So I checked the integrity level of every RestDock process event: the launches, the uninstaller, everything.

Integrity level	Meaning	Observed
4096	Low	No
8192	**Medium: a normal user token, not elevated**	**All events**
12288	High: elevated, administrator	No
16384	System	No

Even the uninstaller ran at medium. Uninstallers of this installer type inherit the privilege setting of the installer that built them, so the package was built to install **without administrator rights**. Least privilege did not stand in its way at all.

8.2 Who installed it?

The data cannot say. The install events fall outside the 30-day retention window, and a program started by the user and one started by a helpdesk technician working inside the user's session look identical from the endpoint. I kept the report neutral on this point and listed the helpdesk ticket history as the only way to settle it. It is not a question a query can answer.

> **Lesson:** say exactly what the data proves and stop there. "Does not need admin" is proven. "The user installed it" is not, and writing it would put a claim about a person in the report with nothing behind it.

9. Hunt 2: Bundled in a Free Installer

Two weeks later I was hunting through a low-volume alert category in the SIEM and found another Command and Control alert, this time for `relaynode_agent.exe`, signed by a residential proxy company. The Case 1 method applied directly, with one new twist each step.

Question	Answer	How
How did it arrive?	Inside a free media codec pack downloaded through the browser. The proxy installer ran silently as a child of the codec setup	Process tree from the download to a silent install flag
With what rights?	Elevated, under a **local administrator account**, about ten seconds after the user launched the setup	Username on the process events changed mid-chain, consistent with a UAC prompt
How does it persist?	A scheduled task that starts the agent at every boot	Task creation command in the process events
Is it really forwarding?	Yes: about 300 distinct destinations in two and a half days, upload and download almost equal (~45 MB each way)	Per-destination bytes from the network view
Whose traffic?	Alerts showed browser User-Agents from many Android phone brands going to an advertising domain	Alert descriptions

The User-Agents are the clearest proof I found in any of the three cases. A Windows laptop does not send requests as a dozen different Android phones. Those requests belonged to the proxy network's customers, and the pattern of many mobile identities hitting an ad domain is consistent with **ad fraud** being run through the company's connection.

The symmetry of the bytes is the mirror image of Case 1. There the agent forwarded responses upstream, so outbound dominated. Here the agent fetched pages from websites and passed them to proxy relays, so the volume in from websites roughly equals the volume out to relays. Different products, same job.

One more detail changed the severity. An earlier incident on the same laptop had been closed as a known issue after a malware scan came back clean. A signed PUA survives a malware scan by design, so the clean scan did not answer the question, and alerts kept coming after the closure. I rated this one **High**: elevated install, persistence, live third-party traffic, and an earlier closure that had left it running.

10. Hunt 3: Straight from the App Store

The third alert pointed at `ClipCast.exe`, a screen recorder installed from the **official Windows app store**. The recorder worked: the user had real recordings on disk. It just had a second job that was not in the description.

This case taught the most important query lesson of the series. Querying network events by the app's hash returned **four rows**: DNS to the internal server, and two public IPs. That looks harmless. Querying **DNS query names** for the same hash returned **hundreds of domains**.

```
dataset = xdr_data
| filter actor_process_image_sha256 = "<sha256>"
| filter dns_query_name != null
| comp count() as queries, min(_time) as first_seen by dns_query_name
| sort desc queries
```

Domain group	Why it does not fit a screen recorder
A proxy provider's connectivity test domain	Used by proxy software to confirm the exit node works
Several "what is my IP" services	Proxy nodes report their public IP and location to the network
Hundreds of unrelated sites: social media, shopping, ad trackers	A recorder has no reason to resolve any of them
A few newly registered API domains	Candidate control servers for the proxy SDK

The rule I wrote down for myself: **start from what the app is for.** Write one sentence about what the app does and who it should talk to. A screen recorder should talk to its vendor for updates and licensing, rarely. Anything beyond that sentence is the finding.

Scoping corrected me once more. I first thought the app was new, because it first ran on the alert day. The fleet-wide hash and path search showed an **older version of the same package** on the same laptop about three weeks earlier, and a short run on a second laptop under the same user account. Two hosts, one account.

11. The Pattern Across Three Cases

	Case 1	Hunt 2	Hunt 3
Delivery	Standalone installer, origin unknown	Bundled in a free codec pack	Official app store
Rights	Medium, no admin needed	Elevated with local admin	Store app, user context
Persistence	Registry autostart value	Scheduled task	Store app relaunch
Best proof	Outbound ~10x inbound	Android User-Agents from a Windows laptop	Hundreds of unrelated DNS names
Endpoints	2	1	2 (one account)
Alert to case?	No	Closed early as known issue	Found by hunting

Three different products, three delivery routes, and one shared outcome: a company laptop became somebody else's exit to the internet. The most useful thing I learned is that the endpoint control was not the weak point. The EDR saw all three. **The alerts did not reach a human through the normal queue.** Case 1's second laptop ran for two weeks in that gap.

That is why the first recommendation in this report is about the alert pipeline, not about proxyware. Removing three programs closes three cases. Fixing the pipeline closes the next one before it starts.

12. Verdict, Containment and Recommendations

True Positive, PUA / proxyware, on all three cases. No sign of credential theft, lateral movement, data staging or a second payload in any of them: the process trees stayed flat, no child processes outside the

products' own components, and no new logons tied to them. The harm is exposure: company bandwidth and the company's public IP rented to unknown third parties.

Network isolation would have been out of proportion for software of this kind. The response was removal plus blocking.

12.1 Containment and eradication (done)

- All three programs uninstalled from every affected endpoint. Autostart value, scheduled task and store package removed.
- File hashes blocked in the EDR so the same builds cannot run again.
- Fleet-wide hash and path searches re-run after removal: no remaining instances.

12.2 Recommendations

1. Ensure every EDR alert in the Command and Control category creates a case in the case management platform, and review why these did not.
2. Do not close a PUA alert on the strength of a clean malware scan. Check whether the process is still running and still connecting.
3. Add a scheduled hunt: non-browser processes whose outbound bytes far exceed inbound, and non-browser processes resolving proxy test or IP-check domains.
4. Restrict app store installs and downloaded installers to an approved list, and review local administrator accounts on laptops.
5. Block known proxyware families by publisher and hash, not by destination IP. Their relay addresses rotate and their health checks use major cloud providers.

13. Indicators and ATT&CK Mapping

Indicator	Notes
`RestDock.exe` in per-user `AppData\Local\Programs` (fictionalized)	Case 1 proxyware. Registry autostart
`relaynode_agent.exe` plus a scheduled task of the same name (fictionalized)	Hunt 2 proxyware, bundled in a codec pack
`ClipCast.exe` in a store package folder (fictionalized)	Hunt 3 screen recorder with proxy SDK
Non-browser process, outbound bytes many times inbound	Exit-node behavior, Case 1
Non-browser process resolving proxy test and IP-check domains	Exit-node self-test, Hunt 3
Mobile User-Agents in traffic from a Windows endpoint	Third-party traffic through the host, Hunt 2

Hashes and relay IPs are omitted in this edition. In the original reports they were listed, with the note that relay IPs rotate and should not be the main control.

Tactic	Technique	Confidence
Impact	T1496.002 Resource Hijacking: Bandwidth Hijacking	Strong: measured volume and third-party traffic in all three cases
Persistence	T1547.001 Registry Run Keys / Startup Folder	Strong for Case 1: autostart value removed at uninstall
Persistence	T1053.005 Scheduled Task	Strong for Hunt 2: task creation seen in process events
Execution	T1204.002 User Execution: Malicious File	Medium: Hunt 2 started from a user-launched installer that carried the PUA

14. Limitations and Lessons Learned

14.1 Limitations

- **Retention.** Case 1 started before the 30-day window. First-seen dates are lower bounds and the install events are gone.
- **Sampled and incomplete network telemetry.** Volumes are what the EDR recorded. Real totals are at least as large.
- **Who installed Case 1.** Not answerable from endpoint data. Needs the helpdesk history.
- **Content of third-party traffic.** Visible only in part (alert User-Agents, DNS names). The reports do not claim to know everything the proxy customers did.

14.2 Lessons Learned

1. **Check your own report like someone else wrote it.** The verdict survived; four of its supporting claims did not. Re-querying before sharing is cheaper than correcting after.
2. **Know what your telemetry does not keep.** No successful TCP connects in the raw dataset changed which view I had to use. An empty result is a fact about the tool first.
3. **Volume is a better witness than geography.** Destinations can look scattered or clustered for many reasons. A 10 to 1 upload ratio on a work laptop has only one.
4. **Query DNS, not just network events.** Four network rows versus hundreds of domain names was the difference between "harmless" and "proxyware".
5. **Start from what the app is for.** One sentence about expected behavior turns a long list of connections into a short list of findings.
6. **Prove only what the data proves.** "No admin needed" was measured. "The user did it" was not, so it stayed out.
7. **Fix the pipe, not only the leak.** The EDR saw every case. The gap was between the alert and a human.